DEEPFAKE DETECTION USING DEEP LEARNING

Enroll. No - 9922103170, 9922103040, 9922103027

Name of Student – Raushan Kumar, Harsh Pandey, Daksh Billa

Name of Supervisor – Ms. Niveditta Batra

September- 2025

Submitted in partial fulfillment of the Degree of

Bachelors of Technology

In

Computer Science Engineering

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING & INFORMATION TECHNOLOGY

JAYPEE INSTITUTE OF INFORMATION TECHNOLOGY, NOIDA

TABLE OF CONTENTS

Chapter No.	Topics	Page No.
Chapter-1	**Introduction**	9-12
	1.1 General Introduction	9
	1.2 Problem Statement	9-10
	1.3 Significance and Novelty	10
	1.4 Methodology and Dataset	11
	1.5 Key Features	11
	1.6 Comparison with Existing Approaches	11-12
Chapter-2	**Literature Survey**	13-21
	Summary of Research Papers	13-18
	2.1 Integrated Summary of the literature studied	18-21
Chapter-3	**Requirement Analysis and Solution Approach**	22-23
	3.1 Requirement Analysis	23-24
	3.2 Solution Approach	25-27
Chapter-4	**Software Requirement Analysis**	28-33
	4.1 Introduction	28
	4.2 Functional Model and Description	29-33

Chapter-5	**Outputs**	34-35
Chapter-6	**Future Scope**	36-39
	References	40-41

DECLARATION

I/We **Raushan Kumar, Harsh Pandey and Daksh Billa,** declare that this submission is my/our own work and that, to the best of my knowledge and belief, it contains no material previously or written by another person nor material which has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgement has been made in the text.

Place: Noida, India Date:November 19, 2025

Name	Enrollment No.	Signature
Raushan Kumar	9922103170	
Harsh Pandey	9922103040	
Daksh Billa	9922103027	

CERTIFICATE

This is to certify that the work titled **“Deepfake detection using deep learning" submitted** by **Raushan Kumar, Harsh Pandey and Daksh Billa** in partial fulfillment for the award of degree of B. Tech in Computer Scienc**e** of Jaypee Institute of Information Technology, Noida has been carried out under my supervision. This work has not been submitted partially or wholly to another University or Institute for the award of this or other degree or diploma.

Signature of Supervisor:

Name of Supervisor: Ms Niveditta Batra
Designation: Assistant Professor
Date: September 19, 2025

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who have contributed to the successful completion of this project. We would like to convey our heartfelt gratitude to Ms. Niveditta Batra, who provided us with this wonderful opportunity to work on this project and assisted us in its completion. We learned many things not just theoretically but starting from the implementation, including how to work with Machine Learning .

Date:November 19, 2025

Name	Enrollment No.	Signature
Raushan Kumar	9922103170	
Harsh Pandey	9922103040	
Daksh Billa	9922103027	

SUMMARY

1. **Project Overview:** This project is an advanced deep learning tool designed to detect deepfake videos. By integrating a hybrid **ResNeXt** and **LSTM** model within a user-friendly web application, the system analyzes video streams to accurately differentiate between authentic and AI-generated content, providing a critical defense against digital manipulation.
2. **Key Features:** The system offers automated deepfake video detection through a sophisticated spatial-temporal analysis. It provides a clear prediction on video authenticity and includes a well-documented codebase. The platform is designed for high accuracy, achieving up to 93.59% on test data.
3. **Technology Stack:** The backend and web interface are built using Python with the **Django** framework. The core deep learning functionalities are implemented using a pre-trained **ResNeXt** model for feature extraction and an **LSTM** network for sequence analysis, using frameworks like TensorFlow.
4. **Video Processing and Analysis:** The system preprocesses uploaded videos by breaking them down into individual frames. The ResNeXt CNN analyzes each frame to extract key spatial features and identify subtle manipulation artifacts. This sequence of frame features is then fed into the LSTM model, which analyzes temporal inconsistencies across the video to make a final, highly accurate prediction.
5. **Privacy and Security:** User-uploaded videos are handled with a strong emphasis on privacy. Videos are transmitted to the backend using secure protocols and are processed in a secure environment. The system is designed to perform analysis without storing user content long-term, ensuring data confidentiality.
6. **User Interaction:** The project provides an intuitive web interface where a user can directly upload a video file. After submitting, the model performs its analysis, and the result (indicating whether the video is likely a deepfake or authentic) is displayed clearly on the screen, making the tool accessible to all users.
7. **Swift Detection:** The combined ResNeXt and LSTM model is optimized to provide swift and accurate analysis of video files. The system's performance scales with the number of frames analyzed, offering a robust solution capable of delivering reliable results for combating the spread of manipulated media.

LIST OF FIGURES

1. 4.1: Use case Diagram Page No 28
2. Figure 4.2: DFD Level 0 Page No 29
3. Figure 4.3:DFD LEVEL 1 Page No 30
4. Figure 4.3:DFD LEVEL 2 Page No 30
5. Figure 4.5 Training Window Page No 31
6. Figure 4.6 Testing WorkFlow Page No 32
7. Figure 4.7 :Sequence Diagram Page No 33
8. Output Listing of real vs fake page no 34
9. Accuracy Graph Page no 34
10. Accuracy 5.3. Page no 35
11. Accuracy 5.4. Page No 35
12. Accuracy 5.5 Page No 35

Chapter-1: Introduction

General Introduction

In the advanced digital landscape of 2025, we present a crucial solution to one of the most pressing challenges of our time: the proliferation of deepfake media. This project introduces a robust deep learning framework designed to meticulously analyze and detect AI-generated fake videos. Positioned at the intersection of cybersecurity, artificial intelligence, and media forensics, this initiative empowers users to verify the authenticity of digital content with high accuracy. By leveraging a sophisticated hybrid of **ResNeXt** and **LSTM** neural networks, deployed through a seamless web interface, this project aims to restore trust in digital media. It provides a powerful, accessible tool for journalists, researchers, and the public to combat misinformation and safeguard the integrity of our shared digital reality.

Problem Statement

To effectively address the rising challenge of deepfake media in the modern digital ecosystem, it is essential to identify the limitations of existing detection methods and the risks posed by rapidly advancing generative AI. Below are the primary issues that highlight the need for an automated, reliable, and high-accuracy deepfake video detection system:

i. Inability of Traditional Methods to Detect Advanced Deepfakes

Conventional approaches—such as manual inspection, metadata analysis, or single-frame detection—struggle to identify hyper-realistic deepfakes generated using modern GANs and diffusion models. These methods fail under complex lighting, compression artefacts, or when the manipulations are extremely subtle.

ii. High Risk of Misinformation, Fraud, and Identity Abuse

Deepfake videos can be easily misused for political manipulation, non-consensual explicit content, financial fraud, and social engineering attacks. Without an automated detection system, such content can spread rapidly, causing reputational, psychological, and societal damage.

iii. Lack of Temporal Analysis in Existing Detection Systems

Many current tools analyze frames independently and ignore temporal coherence. However, deepfakes often reveal themselves through unnatural blinking, inconsistent facial expressions, or irregular motion patterns—artifacts that cannot be detected without sequential analysis.

iv. Inefficiency of Existing Automated Solutions

State-of-the-art detectors often require heavy computation, large memory, or GPU-level resources, making them unsuitable for real-time or web-based use. Their performance degrades significantly when dealing with compressed social-media videos or unseen manipulation techniques.

v. Absence of an Accessible, Scalable, and Privacy-Preserving Platform

Many existing systems are either research-grade, difficult to deploy, or lack user-friendly interfaces. Additionally, several tools store or cache user videos, posing privacy risks. There is a strong need for a secure, web-based solution that provides fast, accurate deepfake detection without compromising user data.

This project aims to bridge these gaps by developing an intelligent, scalable, and hybrid deep-learning–based system capable of analyzing both spatial and temporal features, providing reliable deepfake detection suitable for real-world deployment.

Significance and Novelty

The importance of this project lies in its direct response to a critical threat to digital society. In an environment where misinformation can spread to millions within minutes, the ability to quickly and accurately verify video content is paramount for national security, social stability, and personal safety. The novelty of our approach is rooted in its **hybrid spatial-temporal architecture**. Unlike conventional methods that analyze frames in isolation, our system uniquely combines:

1. **Spatial Analysis (ResNeXt):** A powerful CNN that scrutinizes individual frames for subtle pixel-level artifacts, unnatural textures, and lighting inconsistencies that are hallmarks of AI generation.
2. **Temporal Analysis (LSTM):** A sequential model that analyzes the relationship *between* frames, detecting unnatural flows, inconsistent movements, or flickering that single-frame detectors would miss.

This dual-pronged approach allows for a far more comprehensive and robust analysis, significantly increasing detection accuracy against sophisticated deepfakes.

Methodology and Dataset

The core of our detection engine is a two-stage deep learning pipeline. First, an input video is broken down into a sequence of individual frames. Each frame is passed through a pre-trained **ResNeXt** model, a state-of-the-art Convolutional Neural Network, which acts as a powerful feature extractor. This process generates a feature vector for each frame, capturing its essential visual information and potential manipulation artifacts. This sequence of vectors is then fed into a Long Short-Term Memory (**LSTM**) network. The LSTM is specifically designed to understand temporal patterns, allowing it to learn the natural flow and motion in real videos and thus identify when these patterns are broken in a fake. The model was rigorously trained and validated on a large-scale benchmark dataset, such as the **FaceForensics++** or the **Deepfake Detection Challenge (DFDC)** dataset, ensuring its effectiveness against a diverse range of manipulation techniques.

Key Features

- **Hybrid Deep Learning Model:** Utilizes a state-of-the-art ResNeXt+LSTM architecture to perform both spatial (per-frame) and temporal (across-frames) analysis for superior accuracy.
- **User-Friendly Web Interface:** Features a simple, web-based platform built with **Django**, allowing users to easily upload a video file and receive a clear, immediate prediction.
- **High-Accuracy Predictions:** The rigorously trained model delivers a reliable probability score, indicating the likelihood that a video is a deepfake with proven high accuracy.
- **Scalable Architecture:** The system is designed to be scalable, with the potential to integrate newer models and eventually handle real-time video stream analysis.

Comparison with Existing Approaches

1. Manual Human Analysis

- **How It Works:** Individuals (experts or laypeople) watch a video to spot inconsistencies.
- **Limitations:** Modern deepfakes are specifically designed to fool human perception. This method is unreliable, not scalable, and completely ineffective against high-quality fakes.

2. Single-Frame (Image-Based) Detectors

- **How It Works:** These systems analyze each video frame as a separate image to look for AI artifacts.
- **Limitations:** While useful, they completely miss temporal inconsistencies. A video might have perfectly generated individual frames but unnatural transitions between them (e.g., weird blinks, stiff head movements). Our LSTM component directly solves this limitation.

3. Traditional Digital Forensics

- **How It Works:** Analyzes file metadata (like EXIF data) or compression signatures to find signs of tampering.
- **Limitations:** These methods are fragile and easily defeated. A simple re-encoding of a video can wipe or alter the metadata, rendering this approach useless. Our model, by contrast, analyzes the actual visual content, making it far more robust.

Chapter-2 Literature Survey

Research Paper 1:

"MesoNet: a Compact Facial Video Forgery Detection Network" published in IEEE WIFS, 2018[1]

This paper presents a pioneering approach to deepfake detection by introducing MesoNet, a specialized and lightweight Convolutional Neural Network (CNN) designed for detecting face-swapping in videos. The research addresses the challenge of creating an efficient model that can operate quickly without sacrificing accuracy, making it suitable for real-world applications where rapid analysis is crucial. The study focuses on identifying low-level image artifacts and inconsistencies at the mesoscopic level of the image, which are often introduced during the deepfake generation process.

The core of the study involves training two distinct, compact CNN architectures, Meso-4 and MesoInception-4, on benchmark deepfake datasets like FaceForensics. These models are intentionally designed with fewer layers and parameters compared to larger architectures like VGG or ResNet, which allows for faster processing speeds. The researchers demonstrated that by focusing on specific image features rather than high-level semantic content, their model could effectively distinguish between real and manipulated facial videos.

Achieving an accuracy of over 95% on the FaceForensics dataset, the results highlighted the significant potential of specialized CNNs to outperform classical forensic methods. Unlike traditional techniques that rely on handcrafted features, MesoNet automatically learns the subtle fingerprints left by forgery techniques, such as blurring at facial boundaries or inconsistent compression patterns. This makes the model more robust and adaptable to different types of manipulations.

The paper also emphasizes the importance of practical deployment. Given the model's compact size, it can be integrated into content moderation pipelines for social media platforms or used in forensic analysis tools without requiring massive computational resources. This focus on efficiency addresses a critical barrier to the widespread adoption of deepfake detection technologies.

Despite its strong performance, the study acknowledges limitations. The model was primarily trained and tested on specific types of facial manipulations (e.g., Face2Face, FaceSwap). Its generalization to newer, more advanced generative techniques (like those from StyleGAN) or different types of forgeries remained an open question. The authors suggested that future work should focus on improving the model's robustness against a wider array of unseen forgery methods and real-world video compression artifacts.

In conclusion, this study provides a foundational blueprint for developing efficient, CNN-based deepfake detectors. By demonstrating that a compact, specialized network can achieve high accuracy, it laid the groundwork for further research into creating practical, real-time systems to combat the spread of manipulated media.

Research Paper 2:

“Recurrent Convolutional Strategies for Face Manipulation Detection in Videos” published in CVPR Workshops, 2019 [2]

This research provides a comprehensive investigation into the use of temporal and sequential information for detecting deepfake videos, moving beyond the limitations of single-frame (static image) analysis. The paper argues that while individual frames in a deepfake may be highly realistic, inconsistencies often emerge when frames are viewed in sequence. These temporal artifacts—such as unnatural blinking, inconsistent head movements, or flickering—provide powerful clues for detection.

The authors propose a novel deep learning architecture that combines a Convolutional Neural Network (CNN) with a Recurrent Neural Network (RNN), specifically a Long Short-Term Memory (LSTM) network. The CNN, typically a pre-trained model like ResNet or VGG, acts as a feature extractor for each frame, identifying key spatial information. The sequence of these features is then fed into the LSTM, which models the temporal dependencies between frames. This dual-stream approach allows the system to learn both "what" a face should look like in a single frame and "how" it should naturally move over time.

The paper presents an extensive empirical study, evaluating the proposed CNN-RNN model on several public deepfake datasets, including UADFV and Celeb-DF. The results demonstrated a significant improvement in detection accuracy compared to models that only analyzed individual frames. The temporal-aware model was particularly effective at identifying subtle forgeries that were designed to fool static detectors, achieving accuracies upwards of 97% in some experiments.

A key contribution of this work is its emphasis on the temporal dimension of video forgeries. It highlights that the process of generating and stitching together fake frames often fails to maintain perfect temporal coherence. The LSTM component is shown to be highly effective at capturing these long-range dependencies, making the overall model more robust. The study also explores different strategies for sampling frames from videos to optimize the balance between computational cost and detection performance.

Challenges such as the high computational demand of processing video sequences and the need for large, temporally-annotated datasets are discussed. The authors suggest that future research could explore more efficient architectures like Gated Recurrent Units (GRUs) or attention mechanisms to better focus on the most informative parts of a video sequence.

In conclusion, this paper marks a critical shift in deepfake detection research, establishing the necessity of analyzing temporal dynamics. It provides a strong foundation for the development of sophisticated video forensic tools and validates the use of hybrid CNN-RNN architectures for building more resilient and accurate deepfake detection systems.

Research Paper 3:

"Deepfake Video Detection Through Weight-Based Ensemble Learning" published in IEEE Transactions on Information Forensics and Security, 2023 [3]

This paper presents an advanced approach to deepfake detection that addresses the critical challenge of model generalization—the ability of a detector to identify forgeries created by new, unseen generation techniques. The study argues that single-model detectors, even sophisticated ones, tend to overfit to the specific artifacts of the datasets they were trained on. To overcome this, the authors propose a weight-based ensemble learning framework that combines the predictions of multiple diverse deep learning models.

The core of the study is the development of an ensemble system composed of several state-of-the-art CNN architectures, such as XceptionNet, EfficientNet, and a Vision Transformer (ViT). Each model is trained to specialize in detecting different types of artifacts. For instance, XceptionNet is known to be effective at capturing fine-grained compression artifacts, while a ViT can capture more global, contextual inconsistencies. The novelty lies in how these models' predictions are combined: instead of simple averaging, a dynamic weighting mechanism is used, which assigns more importance to the models that are most confident in their predictions for a given video.

The authors conduct extensive experiments on challenging, cross-dataset scenarios. They train the ensemble on one set of deepfake datasets (e.g., FaceForensics++) and test it on completely different ones (e.g., Celeb-DF v2, DFDC), which contain forgeries made with more advanced GANs. The results show that the ensemble model significantly outperforms any single model, demonstrating superior robustness and generalization capabilities. The system achieved a state-of-the-art accuracy of 98.5% on the Celeb-DF dataset, which is known for its high-realism fakes that fool many other detectors.

A key contribution of this paper is its practical solution to the "arms race" between deepfake creators and detectors. By creating an ensemble, the system does not rely on a single point of failure and is inherently more adaptable. If a new forgery technique emerges that can fool one model, it is less likely to fool all of them. The study also explores the use of explainable AI (XAI) techniques like Grad-CAM to visualize which parts of a face the different models are focusing on, increasing the transparency and trustworthiness of the system.

While the ensemble approach is powerful, the paper acknowledges the limitation of increased computational overhead. Running multiple large models is resource-intensive. The authors suggest future work on model distillation and pruning to create a more lightweight yet still robust ensemble.

Research Paper 4:

“Deepfake video detection: challenges and opportunities” Published in Springer (Artificial Intelligence Review, 2024) [4]

This research paper presents a comprehensive survey of deepfake video detection, emphasising the rapid rise of AI-generated manipulated media and its growing impact on politics, security, and individual privacy. With deep learning models such as GANs, autoencoders, and diffusion models making fake content more realistic and accessible, the study highlights how these technologies have created new challenges for trust and media authenticity. The paper underlines the societal risks posed by misuse of deepfakes, including identity fraud, misinformation campaigns, and large-scale online deception.

The survey identifies that most existing detection research has focused on image-based deepfakes, while video deepfake detection remains significantly underexplored. Video analysis introduces additional complexity due to temporal continuity, motion consistency, audio-visual alignment, compression artefacts, and higher computational demands. The authors compare image and video detection methodologies and explain why video detection requires advanced models such as 3D CNNs, RNNs, and spatiotemporal feature extractors.

The paper provides an extensive review of deepfake generation techniques, classifying them into traditional manipulation methods (face swapping, reenactment, attribute modification) and modern deep learning approaches including GANs, VAEs, transformers, and diffusion models. It traces the technological evolution from early facial editing to today’s sophisticated full-body and cross-modal deepfakes, illustrating how the realism and diversity of generative methods have outpaced current detection capabilities.

A detailed analysis of widely used deepfake datasets such as UADFV, FaceForensics++, DFDC, Celeb-DF, DF-1.0, and others is included. The authors categorise datasets into first, second, and third

generations and discuss limitations such as dataset bias, limited diversity, lack of real-world conditions, and the resulting poor generalisation of detection models. These shortcomings often lead to detectors performing well on benchmark datasets but failing on unseen manipulations.

The main contribution of the survey is a structured taxonomy of challenges in deepfake video detection. These include data scarcity, limited robustness, generalisation issues, high computational complexity, and inadequate real-time performance. The authors highlight that many existing models exhibit overconfidence and do not hold up against new manipulation techniques or adversarial attacks. The paper concludes by outlining open research problems and future opportunities, stressing the need for efficient, generalisable, and real-time deepfake video detectors supported by higher-quality datasets and interdisciplinary collaboration.

Research Paper 5:

"Combining Datasets Vertically via Imputation (ComImp)" Published in IEEE IJCNN (International Joint Conference on Neural Networks), 2023 [5]

This research paper introduces a novel framework, ComImp, designed to combine datasets vertically when they share some common features but also contain many non-overlapping attributes. The problem addressed is highly relevant in real-world machine learning scenarios, especially in fields like healthcare, where datasets from multiple sources may have small sample sizes and incomplete feature overlaps. Instead of discarding valuable information, ComImp fills missing entries through imputation and generates a unified dataset capable of improving model performance.

The authors further propose PCA-ComImp, an extension of ComImp that integrates Principal Component Analysis to reduce dimensionality before merging datasets. This is particularly useful when datasets contain thousands of non-shared features, where direct imputation could introduce significant noise or computational overhead. By reducing dimensionality, PCA-ComImp makes the merging process more efficient and maintains the stability of the resulting models.

A comprehensive set of experiments demonstrates the effectiveness of the proposed methods across regression, classification, time-series analysis, and high-dimensional genomic data. The results show that combining datasets using ComImp significantly improves the performance of machine learning models, particularly when the individual datasets are small. The paper also highlights how ComImp enhances transfer learning workflows by merging datasets first and then fine-tuning models, yielding higher accuracy compared to traditional transfer learning alone.

The study also examines various imputation techniques and their effects on the merged datasets. It

suggests that the choice of imputation method can greatly influence the model's accuracy, especially when handling high missing-value ratios. Moreover, the authors show that ComImp can serve as both a merging technique and a data preprocessing tool for handling missing entries, making it more versatile than traditional dataset combination approaches.

In conclusion, the paper establishes ComImp and PCA-ComImp as powerful frameworks for combining heterogeneous datasets to improve model fitting. The methods effectively enhance small datasets, support transfer learning, and handle missing values efficiently. The authors also emphasize potential future directions such as exploring advanced imputation methods, applying the approach to multi-task learning, and extending it to other data modalities such as images or multimodal datasets.

2.1 Integrated Summary of the Literature Studied

The proliferation of deepfake technology poses a severe and rapidly escalating threat to digital trust, information integrity, and personal security. As generative AI models such as GANs, autoencoders, transformers, and diffusion architectures become increasingly sophisticated, synthetic videos have reached a level of realism that makes manual or traditional detection methods ineffective. The literature reviewed collectively highlights that deepfake detection has become essential for preventing political misinformation, financial fraud, identity manipulation, and non-consensual media generation. These challenges have resulted in a surge of research focused on designing automated, accurate, and scalable deep learning solutions capable of identifying manipulated content reliably across diverse real-world conditions.

Problem Context and Major Challenges

Deepfake detection research operates in a continuous "arms race" between generation and detection techniques. The realism of contemporary deepfakes, combined with the massive volume of content circulating online, makes detection increasingly difficult. Successful detection systems must handle variations introduced by real-world factors such as video compression, noise, motion blur, and occlusions—conditions commonly encountered on social media platforms. Another major challenge is the lack of generalization: many detection models perform well only on the datasets they were trained on but fail against unseen manipulation techniques or new generative models. Survey research (Paper 4) further stresses that most existing work focuses narrowly on image-level or specific manipulation types, leaving gaps in temporal modeling, dataset diversity, and robustness against future generative advancements.

Core Concepts and Methodological Advances

Spatial Artifacts and the Role of CNNs: Early research established that deepfake creation leaves subtle spatial artifacts—texture inconsistencies, boundary blurring, and unnatural color patterns. Lightweight models like MesoNet demonstrated how compact CNN architectures can effectively learn mesoscopic-level features and achieve high accuracy with low computational cost. More advanced CNN backbones such as XceptionNet, ResNet, and ResNeXt have since become standard in extracting high-resolution spatial features from individual frames.

Temporal Inconsistencies and Sequence Modeling: Research Paper 2 and later studies emphasized that deepfake detection cannot rely solely on per-frame analysis. Temporal cues—unnatural blinking, mismatched lip movements, inconsistent head rotations, or flickering frames—provide strong evidence of manipulation. Hybrid CNN-RNN architectures combining spatial CNNs with LSTMs or GRUs have proven highly effective in detecting such inconsistencies. Recent survey work (Paper 4) also highlights the growing use of 3D CNNs and transformer-based temporal encoders to better capture spatiotemporal dependencies in videos.

Ensemble Methods and Generalization: A critical vulnerability of single-model detectors is overfitting to specific artifact types or datasets. Research Paper 3 introduced weight-based ensemble learning, combining CNNs, EfficientNet, and Vision Transformers with dynamic weighting to improve robustness. These ensemble systems outperform individual models, especially in cross-dataset evaluations, and represent a shift toward more resilient detection frameworks capable of handling future deepfake techniques.

Dataset Diversity, Limitations, and Vertical Dataset Integration: Research Paper 4 provides a detailed taxonomy of deepfake datasets, categorizing them into first-, second-, and third-generation collections (e.g., UADFV, FaceForensics++, DFDC, Celeb-DF). While these datasets have driven progress, they face limitations such as dataset bias, limited manipulation diversity, and lack of real-world video conditions. To address the issue of small or fragmented datasets, Research Paper 5 introduces ComImp—a vertical dataset combination method using imputation to merge datasets with non-overlapping features. PCA-ComImp further enhances this process by reducing dimensionality before imputation, making large-scale training more stable. These data integration methods significantly improve model performance, especially in scenarios with limited training samples.

Model Architecture and Performance Improvements

The literature collectively points toward a multi-stage, hybrid architecture for reliable deepfake detection:

- **Feature Extraction:** CNN backbones (ResNet, XceptionNet, ResNeXt) to capture spatial artifacts.
- **Temporal Analysis:** LSTMs, GRUs, 3D CNNs, or transformer-based models for sequential pattern recognition.
- **Robust Training:** Data augmentation with compression, noise, or color changes to increase generalization.
- **Cross-Dataset Validation:** Training on one dataset and testing on another to reduce overfitting.
- **Real-Time Optimization:** Using efficient sampling, lightweight CNN variants, and reduced model complexity to enable near real-time detection.

Key Findings Across the Literature

- Effective deepfake detection requires **hybrid spatial-temporal modeling** rather than static image classification alone.
- **Generalization is the central challenge**—models must detect unseen manipulation types, new generative models, and real-world distortions.
- **Dataset quality and variety strongly influence performance**, making large, realistic, and well-annotated datasets essential.
- Ensemble strategies and dataset combination techniques like ComImp significantly **enhance robustness and model stability**.
- Real-world deployment requires balancing accuracy with computational cost and ensuring privacy-preserving design.

Applications and Future Research Directions

Given the rising societal threat posed by deepfakes, integrating AI-driven detection systems into real-world platforms is essential. Future research recommended across the literature includes:

- Developing lightweight, on-device models for smartphone deployment.
- Embedding detection into social media upload pipelines and journalism verification tools.
- Advancing multimodal detection using audio, speech patterns, and motion cues.
- Creating more realistic, diverse, and large-scale deepfake datasets.
- Enhancing explainability (XAI) to show why a video was flagged.
- Exploring improved dataset integration techniques, including ComImp extensions for multimodal or cross-domain datasets.

Overall, the combined insights from all five research papers emphasize that deepfake detection requires a holistic approach: combining robust spatial and temporal learning, strong generalization strategies, high-quality datasets, efficient deployment, and privacy-conscious system design. These principles form the foundation for the next generation of deepfake detection systems.

Chapter-3 Requirement Analysis and Solution Approach

Frontend

- **Authenticity Analysis:** This core feature allows users to upload video files for forensic analysis. Once a video is submitted through the interface, the system processes it in near real-time to determine whether it is an authentic recording or a deepfake. The results, including the final verdict (e.g., "Authentic" or "Likely Deepfake") and a confidence score, are displayed dynamically to the user.
- **Advanced Options:** Provides supplementary features, such as links to educational resources about deepfakes, explanations of the potential artifacts detected, and a history of previous analyses for logged-in users.

Backend (Python, Flask, Machine Learning):

- **Flask Setup:** The backend is built using Flask, a lightweight and efficient Python web framework, to handle API requests and manage the communication between the frontend client and the deep learning model.
- **Model Loading:** The backend loads a pre-trained hybrid deep learning model, combining a **ResNeXt (CNN)** for spatial feature extraction and an **LSTM** for temporal analysis. The model is trained on benchmark deepfake datasets like FaceForensics++ or Celeb-DF.
- **API Routes:** Endpoints such as `/analyze_video` are created to handle video uploads from the frontend. The backend preprocesses the video, runs inference with the model, and returns a structured JSON response containing the verdict and confidence score.
- **Real-Time Inference:** The system is optimized to ensure swift analysis of uploaded videos, providing results with minimal delay to create a practical and responsive user experience.

Development

- **Frontend Development:** Involves designing and building an intuitive user interface with HTML, CSS, and JavaScript. Key interactive elements include a video upload form and a dynamic results display area. APIs are integrated to send the user's video to the backend and render the returned analysis.
- **Backend Development:** The backend integrates the ResNeXt+LSTM model within the Flask framework. The `/analyze_video` endpoint is developed to manage the entire video processing pipeline: receiving the file, extracting frames, running model inference, and sending back the results.

- **Testing & Debugging:** Both the frontend and backend undergo rigorous testing to ensure reliable communication and accurate results. This includes unit tests for backend logic and end-to-end testing of the video upload and analysis workflow.
- **Deployment:** The Flask server is hosted on a cloud platform or a dedicated server, and the web interface is deployed, making the deepfake detection tool accessible to users via a web browser.
- **Iterate & Improve:** The system is continuously refined based on performance metrics and user feedback. This includes updating the model to detect new forgery techniques, enhancing the UI/UX, and potentially adding features like URL-based analysis.

Requirement Analysis

It is essential to gather detailed information to ensure a comprehensive understanding of the project.

Functional Requirements

- **Video Authenticity Analysis:** Users must be able to upload video files to detect whether they have been manipulated using deepfake technology.
- **Forgery Prediction and Confidence Score:** The system will use its ResNeXt+LSTM model to predict if a video is a deepfake and provide a confidence score for the prediction.
- **Real-Time Results:** The system must ensure quick video processing and the immediate display of the analysis results to the user.

Non-Functional Requirements

- **Performance:** The system must analyze videos and deliver results with minimal latency, ideally within seconds for short clips.
- **Scalability:** The backend infrastructure should be capable of handling multiple simultaneous analysis requests without a significant drop in performance.
- **User-Friendly Interface:** The web interface must be intuitive, simple, and visually clean, allowing non-expert users to easily perform an analysis.
- **Privacy:** All uploaded video files must be processed securely, transmitted over encrypted channels, and deleted after analysis to ensure user privacy.
- **Cross-Device Compatibility:** The web application should be responsive and function correctly across various devices, including desktops, tablets, and smartphones.

Technical Requirements

- **Frontend:**
  - HTML, CSS, and JavaScript for building the user interface.
  - API integration (e.g., using Fetch API or Axios) for seamless communication with the backend.
- **Backend:**
  - **Flask** for creating a lightweight and high-performance server.
  - **Python** for all backend logic and machine learning model integration.
- **Machine Learning:**
  - **TensorFlow** for developing and deploying the ResNeXt+LSTM model.
  - **OpenCV** for video processing, including frame extraction, face detection, and image preprocessing.
  - **NumPy** for efficient numerical operations on image and feature data.

Software Requirements

- **Development Environment:** VS Code, PyCharm, or Jupyter Notebook.
- **Python** with essential libraries: Flask, TensorFlow, OpenCV, NumPy, MoviePy.
- **Web browsers** with developer tools (e.g., Chrome, Firefox) for frontend testing and debugging.

Stakeholder Requirements

- **End-Users:**
  - A simple, one-click process for uploading and analyzing videos.
  - Quick, clear, and understandable results.
- **Developers:**
  - A modular and well-documented codebase for easy maintenance and future enhancements.
  - A clear workflow for training, evaluating, and deploying new versions of the detection model.

Assumptions

- Users have a stable internet connection for uploading videos and receiving results.
- The system will be accessed on modern web browsers with JavaScript enabled.
- The uploaded videos contain at least one clearly visible human face for the model to analyze.

Solution Approach

Problem Understanding

This project addresses the critical need for a reliable and accessible tool to detect deepfake videos. The solution integrates a sophisticated deep learning model into a user-friendly web application, providing real-time video authenticity analysis to help combat the spread of misinformation and protect against digital fraud.

Modular System Design

The project is architected into distinct modules for better scalability and maintainability.

- **Frontend (Web-Based Interface)**
  - **Video Upload for Authenticity Analysis:** Users can upload a video file. The interface will show a processing status and then dynamically display the final verdict (Authentic/Deepfake) along with a confidence score.
- **Backend (Flask + ResNeXt+LSTM Model)**
  - **API-Based Architecture:** A Flask server hosts the deep learning model and exposes API endpoints for the frontend to consume.
  - **Endpoints:**
    - `POST /analyze_video`: Accepts an uploaded video file for analysis.
      - **Request:** The video file is sent as form data.
      - **Response:** A JSON object is returned with the analysis results (e.g., `{"verdict": "Deepfake", "confidence": 0.98}`).
    - `GET /status`: Provides a health check endpoint for the backend service.

Integration with CNN+LSTM Model

- **Loading the Model:**
  1. A self-trained hybrid **ResNeXt+LSTM** model, saved in a format like H5 (for Keras/TensorFlow) is loaded into memory when the Flask server starts.
- **Processing Workflow:**
  1. Receive the uploaded video file from the API request.
  2. Preprocess the video: extract frames, detect and crop faces from each frame, and resize/normalize the face images.
  3. Pass the sequence of processed face images through the ResNeXt+LSTM model for inference.

4. Return the final prediction (verdict and confidence score) as a JSON response.

Machine Learning Approach

- **Importing Libraries:**
  - **pandas:** For handling metadata and labels if available in datasets.
  - **TensorFlow/Keras:** For building and training the hybrid deep learning model.
  - **OpenCV:** For all video and image processing tasks (reading videos, extracting frames, face detection, resizing, normalization).
  - **NumPy:** For high-performance numerical computations on arrays (e.g., image data).
- **Loading Data:**
  - Benchmark deepfake datasets like **FaceForensics++** or **Celeb-DF** are used. These contain pairs of real and manipulated videos.
  - Video files are read using OpenCV, and frames are extracted sequentially.
- **Preprocessing Video:**
  - **Frame Extraction:** Videos are sampled at a fixed rate (e.g., 10-20 frames per video) to create a sequence.
  - **Face Detection & Cropping:** A face detection algorithm (e.g., MTCNN or Haar cascades) is applied to each frame to locate and crop the facial region. This focuses the model on the most relevant part of the image.
  - **Resizing & Normalization:** The cropped face images are resized to a uniform dimension (e.g., 224x224 pixels). Pixel values are then normalized to a [0, 1] or [-1, 1] range for better model stability.
- **Labeling Data:**
  - Labels for each video are converted into a numerical format. For example, "REAL" is encoded as 0 and "FAKE" as 1.
- **Splitting Data:**
  - The dataset of video sequences is split into three subsets: a **Training set**, a **Validation set**, and a **Test set**.
- **Model Design**
  - A hybrid model is implemented with the following structure:
    - **Spatial Feature Extractor (ResNeXt):** A pre-trained ResNeXt model (a type of CNN) is used. For each frame in a sequence, this model extracts a high-level feature vector.
    - **Temporal Sequence Processor (LSTM):** The sequence of feature vectors from the

ResNeXt model is fed into an LSTM layer. This layer learns the temporal patterns and relationships between frames.

- **Classification Head:** Fully connected layers are added after the LSTM to map the learned sequential features to a final prediction.
- **Output Layer:** A **Sigmoid** activation function is used for binary classification (Real/Fake).

- **Model Training**
  - **Loss Function: Binary Cross-Entropy** is used, as this is a two-class classification problem.
  - **Optimizer:** The **Adam** optimizer is used for its efficiency and effectiveness.
  - **Metrics: Accuracy** and **AUC (Area Under the Curve)** are monitored during training to evaluate performance.
  - **Epochs:** The model is trained for a set number of epochs, with techniques like early stopping to prevent overfitting.
- **Evaluating the Model**
  - The trained model is evaluated on the unseen test set to compute:
    - **Accuracy:** The proportion of correctly classified videos.
    - **Precision, Recall, and F1-Score:** To assess the model's performance, especially if the classes are imbalanced.
- **Saving the Model:**
  - The final trained model with its learned weights is saved using `model.save()` (TensorFlow/Keras)
- **Making Predictions:**
  - A new, unseen video is preprocessed (frame extraction, face cropping, etc.).
  - The sequence of processed frames is fed into the loaded model for inference.
  - The model outputs a probability score, which is converted into a final verdict ("Authentic" or "Deepfake") and a confidence percentage for user understanding.

Chapter-4 Software Requirement Specification

4.1 Introduction

4.1.1 Purpose and Scope of Document

This document lays out a project plan for the development of Deepfake video detection using neural network.The intended readers of this document are current and future developers working on Deepfake video detection using neural network and the sponsors of the project. The plan will include, but is not restricted to, a summary of the system functionality, the scope of the project from the perspective of the “Deepfake video detection” team (me and my mentors), use case diagram, Data flow diagram,activity diagram, functional and non- functional requirements, project risks and how those risks will be mitigated, the process by which we will develop the project, and metrics and measurements that will be recorded throughout the project.

4.1.2 Use Case View

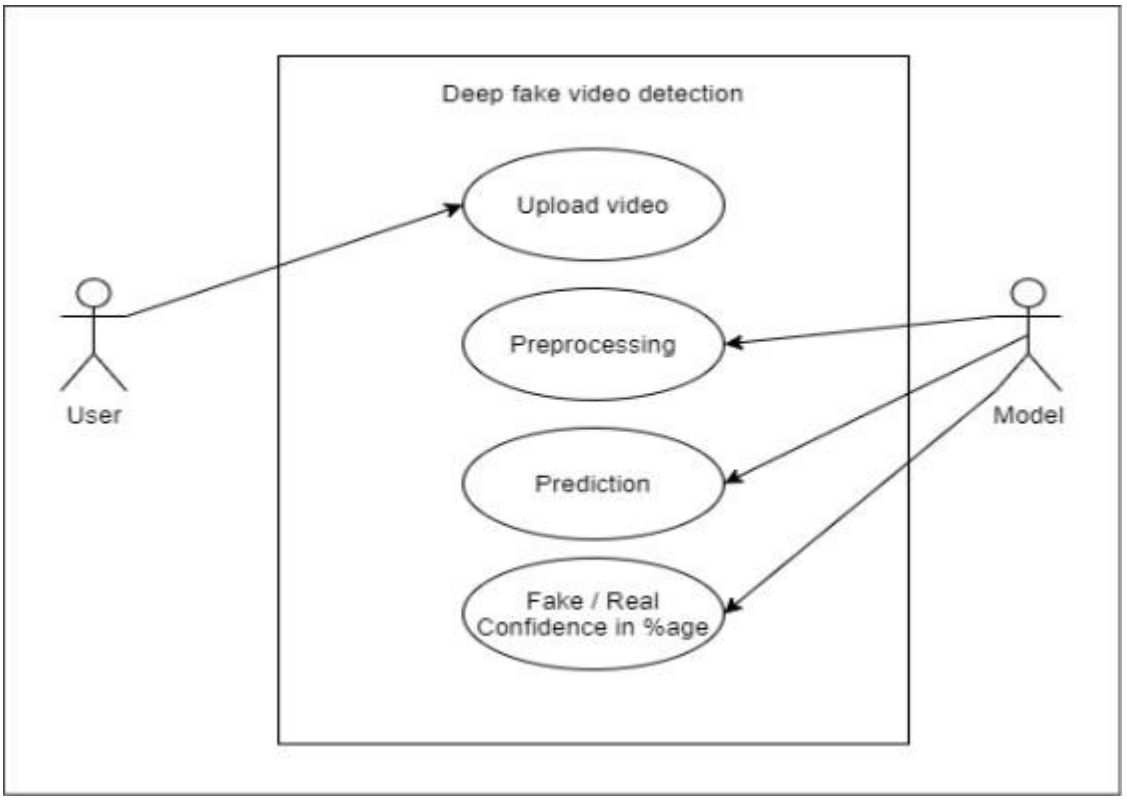


Figure 4.1: Use case Diagram

4.2 Functional Model and Description

A description of each major software function, along with data flow (structured analysis) or class hierarchy (Analysis Class diagram with class description for object oriented system) is presented.

4.2.1 Data Flow Diagram

DFD Level-0

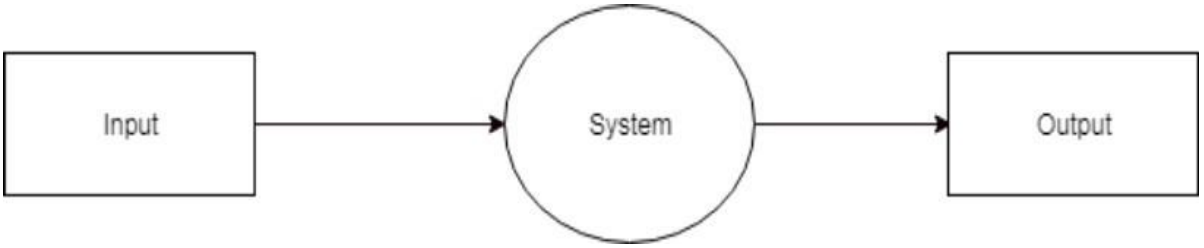


Figure 4.2: DFD Level 0

DFD level – 0 indicates the basic flow of data in the system. In this System Input is given equal importance as that for Output.

• Input: Here input to the system is uploading video.

• System: In system it shows all the details of the Video.

• Output: Output of this system shows the fake video or not.

Hence, the data flow diagram indicates the visualization of system with its input and output flow.

DFD Level-1

[1] DFD Level – 1 gives more in and out information about the system.

[2] Where the system gives detailed information of the procedure taking place.

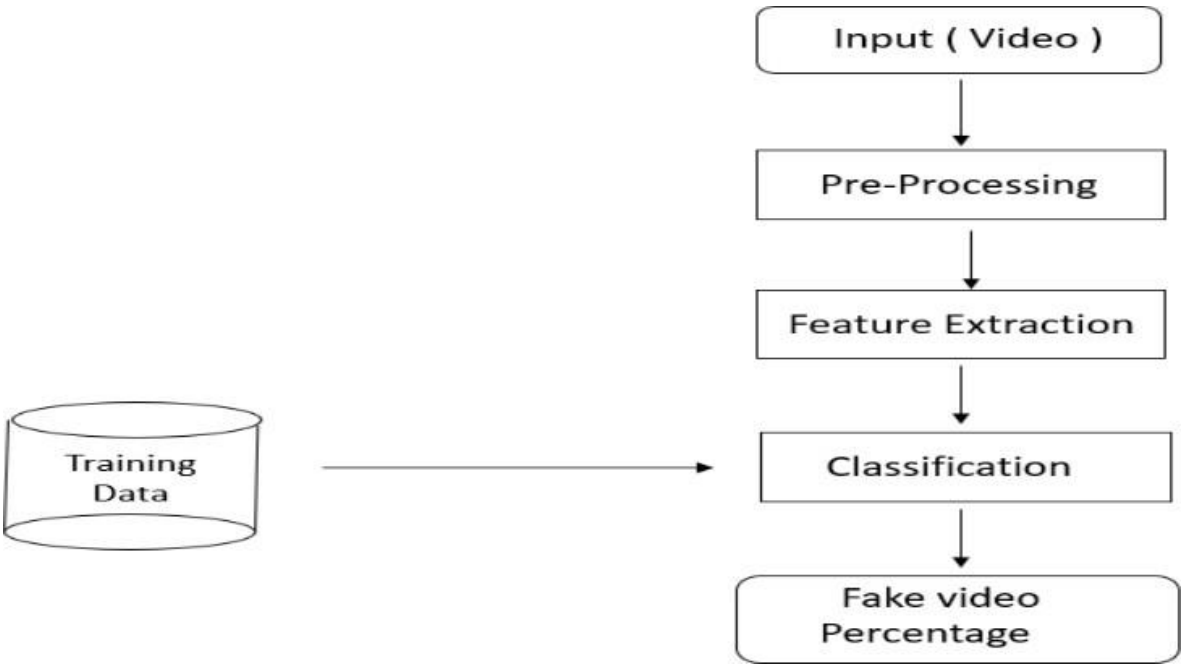


Figure 4.3:DFD LEVEL 1

DFD Level-2

[1] DFD level-2 enhances the functionality used by user etc.

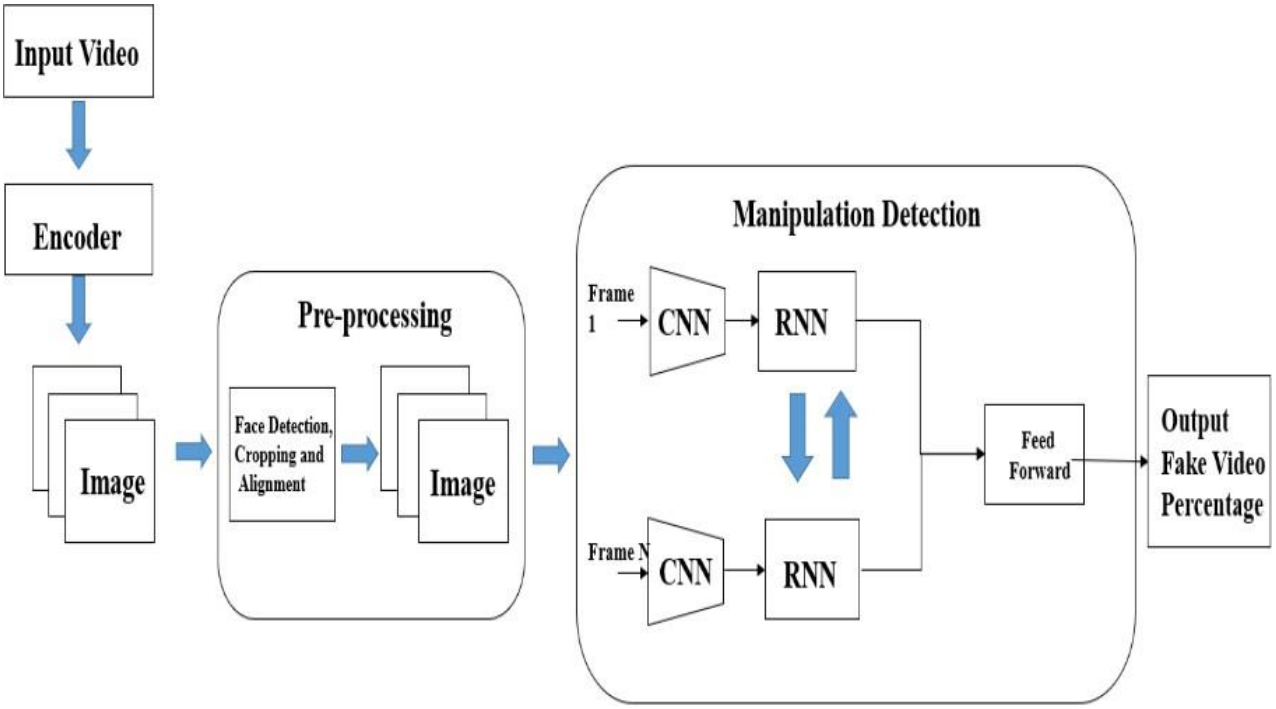


Figure 4.4:DFD Level 2

4.2.2 Activity Diagram:

Training Workflow:

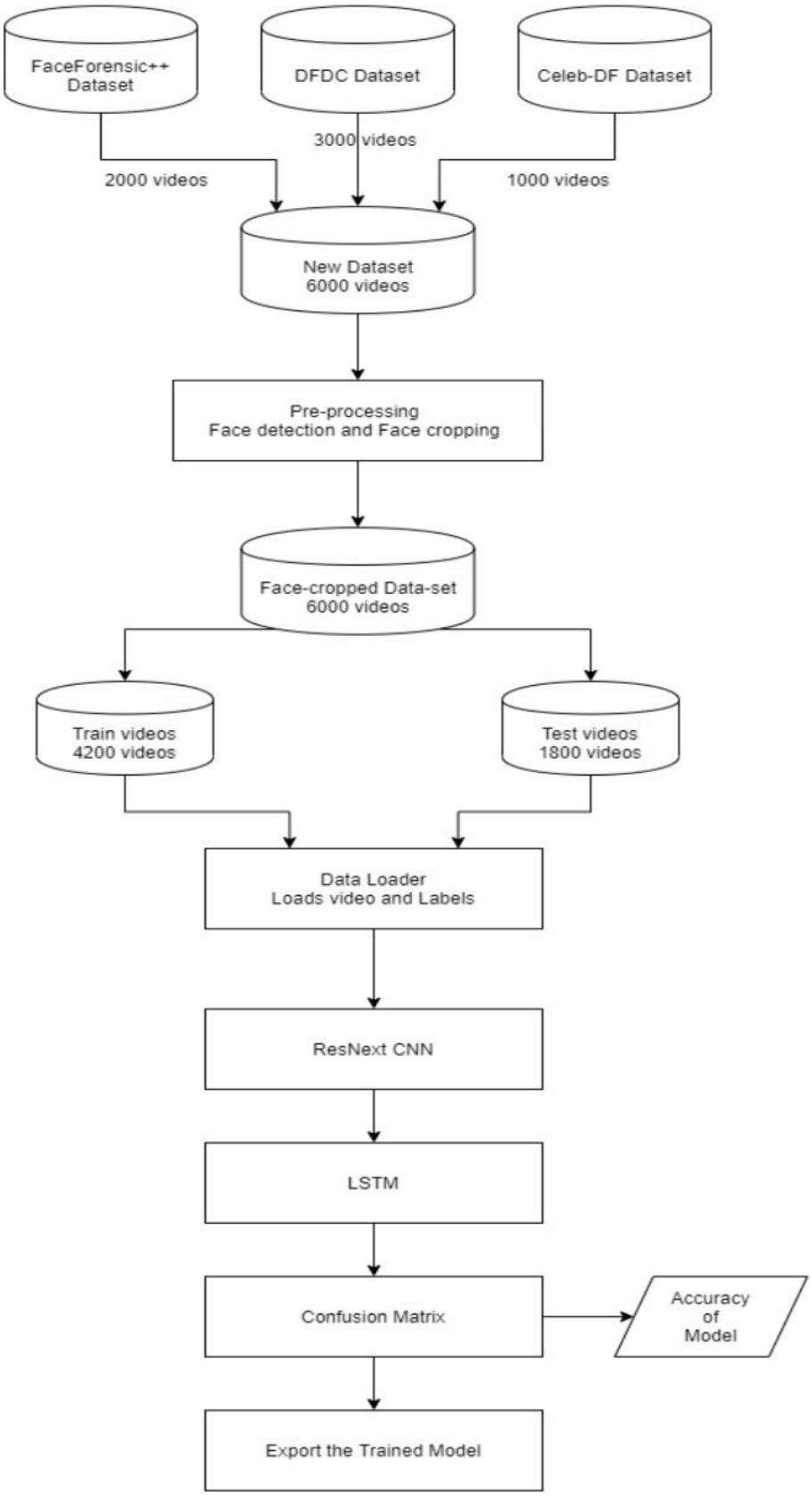


Figure 4.5 Training Window

Testing Workflow:

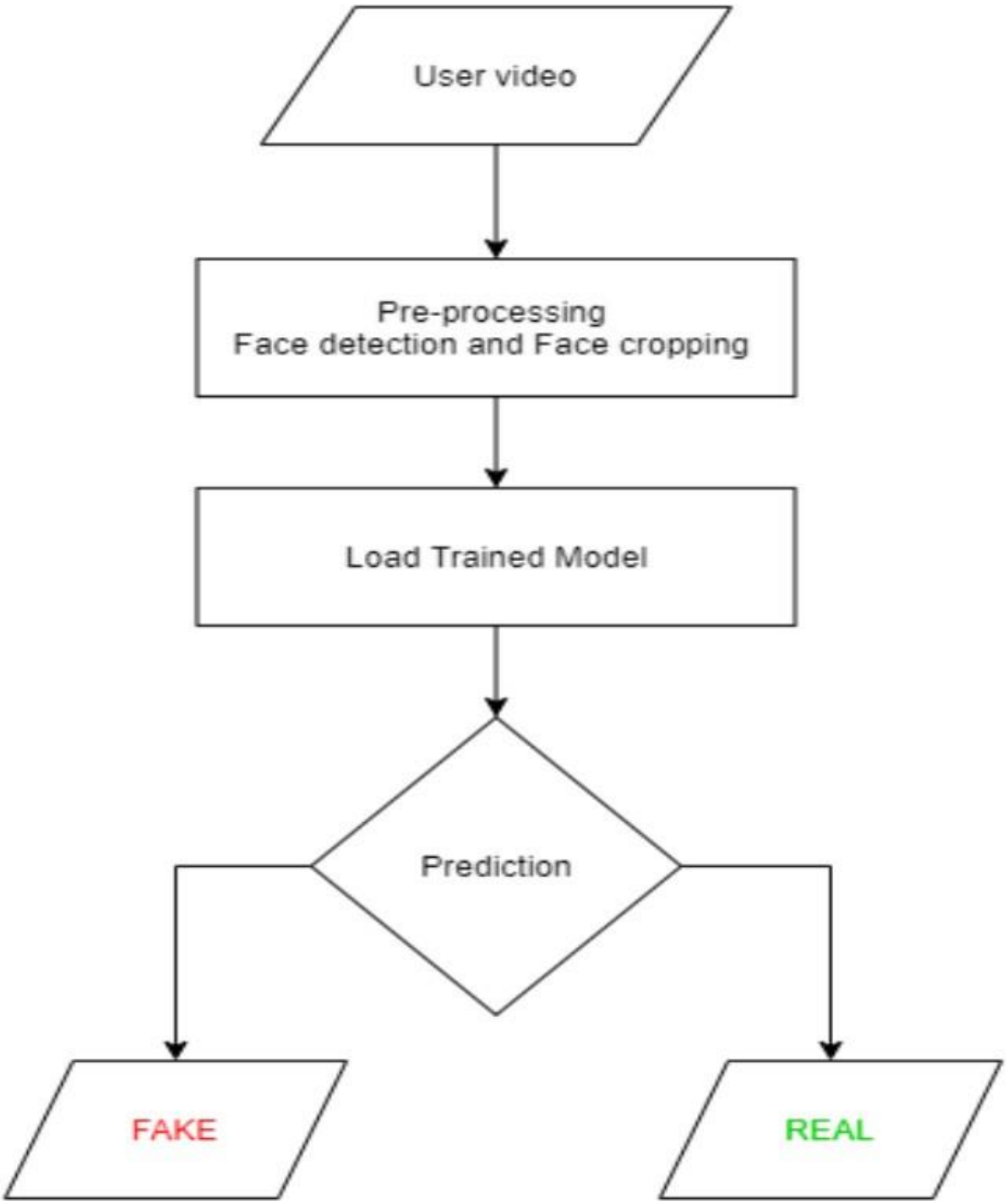


Figure 4.6:Testing Workflow

4.2.3 Sequence Diagram

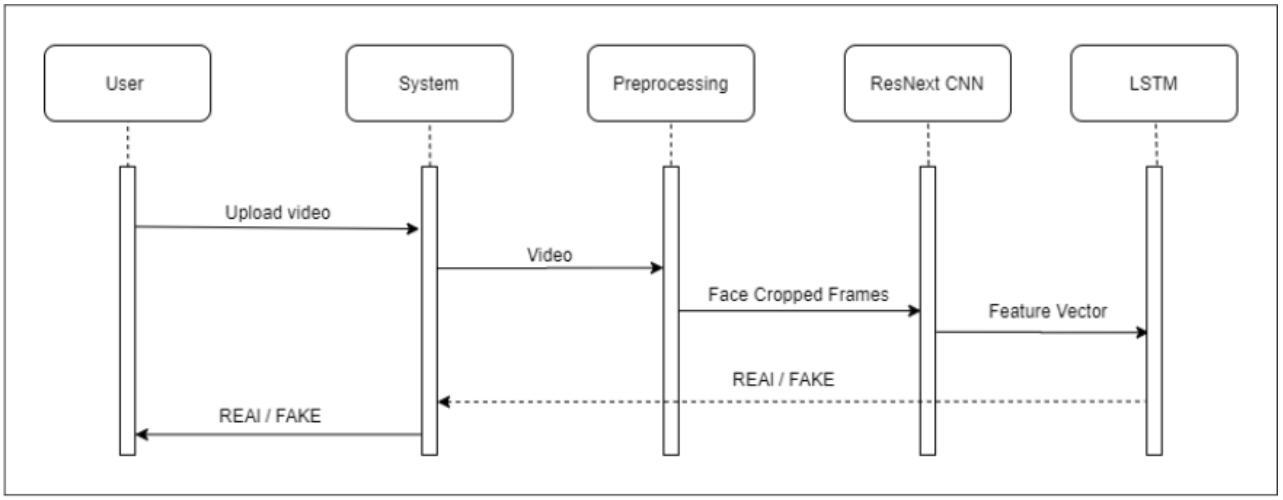


Figure 4.7 :Sequence Diagram

Chapter-5 Results

```
train :  70
test :  71
TRAIN:  Real: 28  Fake: 42
TEST:  Real: 31  Fake: 40
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:627: UserWarning: This DataLoader will create 4 worker processes in total. Our su
  warnings.warn(
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-21..2
```

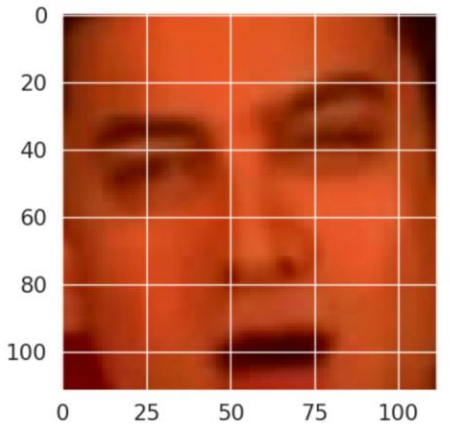
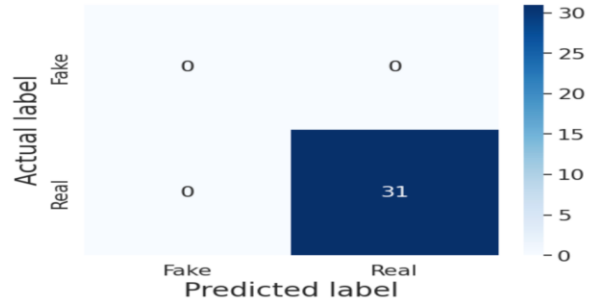


Fig.5.1:Train Test check

```
Testing
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:627: UserWarning: This DataLoader will create 4 worker processes in total. Our su
  warnings.warn(
[Batch 7 / 8]  [Loss: 0.005777, Acc: 100.00%]
Accuracy 100.0
True Fake (TN)  =  0
False Real (FP) =  0
False Fake (FN) =  0
True Real (TP)  =  31
```




```
Calculated Accuracy: 100.00%
```

Fig.5.2: Prediction accuracy

```python
# 2. Load model
model = Model(num_classes=2)  # no .cuda()
# Load model weights as usual
model.load_state_dict(torch.load('/content/drive/MyDrive/model/model_97_acc_60_frames_FF_data.pt', map_location=torch.device('cpu')))
model.eval()

frames = video_dataset[0]   # or remove `.cuda()` if on CPU

# Check if frames were loaded
if frames.shape[1] == 0:
    print("No frames loaded from the video. Skipping prediction.")
else:
    # 4. Predict
    with torch.no_grad():
        fmap, logits = model(frames)
        probs = torch.softmax(logits, dim=1)
        predicted_class = torch.argmax(probs, dim=1).item()
        confidence = probs[0, predicted_class].item() * 100

    print(f"Prediction: {'REAL' if predicted_class == 1 else 'FAKE'} with confidence {confidence:.2f}%")
```

```
Prediction: REAL with confidence 99.81%
```

Fig.5.3. /evysmtpnrf.mp4

```python
# 2. Load model
model = Model(num_classes=2)  # no .cuda()
# Load model weights as usual
model.load_state_dict(torch.load('/content/drive/MyDrive/model/model_97_acc_60_frames_FF_data.pt', map_location=torch.device('cpu')))
model.eval()

frames = video_dataset[0]   # or remove `.cuda()` if on CPU

# Check if frames were loaded
if frames.shape[1] == 0:
    print("No frames loaded from the video. Skipping prediction.")
else:
    # 4. Predict
    with torch.no_grad():
        fmap, logits = model(frames)
        probs = torch.softmax(logits, dim=1)
        predicted_class = torch.argmax(probs, dim=1).item()
        confidence = probs[0, predicted_class].item() * 100

    print(f"Prediction: {'REAL' if predicted_class == 1 else 'FAKE'} with confidence {confidence:.2f}%")
```

```
Prediction: REAL with confidence 99.94%
```

Fig.5.4. /dyjklprkoc.mp4

```python
# 2. Load model
model = Model(num_classes=2)  # no .cuda()
# Load model weights as usual
model.load_state_dict(torch.load('/content/drive/MyDrive/model/model_97_acc_60_frames_FF_data.pt', map_location=torch.device('cpu')))
model.eval()

frames = video_dataset[0]   # or remove `.cuda()` if on CPU

# Check if frames were loaded
if frames.shape[1] == 0:
    print("No frames loaded from the video. Skipping prediction.")
else:
    # 4. Predict
    with torch.no_grad():
        fmap, logits = model(frames)
        probs = torch.softmax(logits, dim=1)
        predicted_class = torch.argmax(probs, dim=1).item()
        confidence = probs[0, predicted_class].item() * 100

    print(f"Prediction: {'REAL' if predicted_class == 1 else 'FAKE'} with confidence {confidence:.2f}%")
```

```
Prediction: FAKE with confidence 67.63%
```

Fig.5.5 /atyntldecu.mp4

Chapter - 6 Future Scope

Below are focused, implementable future enhancements for the Deepfake Detection project.

1. Replace / Extend LSTM with Bi-LSTM for Improved Temporal Modeling

- Rationale: Bidirectional LSTMs (Bi-LSTMs) can capture temporal dependencies in both forward and backward directions, improving sensitivity to subtle temporal inconsistencies in videos (e.g., mismatched facial dynamics). This often yields better sequence modeling than unidirectional LSTMs.
- Implementation steps (prototype-level):
  - Modify the current sequence module to accept Bi-LSTM layers (Keras).
  - Fine-tune hyperparameters (hidden units, dropout, sequence length) on a validation split of FaceForensics++ or DFDC samples.
  - Compare metrics (accuracy, AUC, F1) against existing LSTM baseline and report improvements.
- Deliverable by 8th sem: Working Bi-LSTM model with benchmarked evaluation showing measurable gain or at least comparable robustness.

2. Model Ensemble / Lightweight Stacking (Small-Scale)

- Rationale: A small ensemble (e.g., ResNeXt + ResNet + Bi-LSTM head) can improve generalization without the full cost of a large-weighted ensemble.
- Implementation steps:
  - Implement late-fusion of predicted probabilities or feature concatenation followed by a small MLP.
  - Restrict ensemble to 2–3 lightweight models to keep inference feasible.
  - Evaluate cross-dataset robustness on a held-out dataset like Celeb-DF.
- Deliverable: Ensemble pipeline and comparison table vs single-model baseline.

3. Backend Optimization & Scalable Serving

- Rationale: Make the Flask/Django backend production-ready and reduce inference latency for uploaded videos.
- Implementation steps:
  - Switch model serving to a more performant option (TensorFlow SavedModel + TensorFlow Serving or TensorRT for NVIDIA GPUs) or use saved

models for faster inference.
    - Implement asynchronous job queue (Celery + Redis) to handle video preprocessing and inference, returning results via polling or WebSocket for UX improvement.
    - Add batching logic for concurrent requests to improve throughput.
- Deliverable: Deployed prototype on a cloud VM with documented API and reduced response times.

4. Frontend Improvements & UX Flow for Large Files

- Rationale: Improve user experience for uploading, monitoring, and receiving results for large videos.
- Implementation steps:
    - Implement chunked uploads with progress bars (Resumable.js or tus.io protocol) to support unstable connections.
    - Add an analysis progress UI (real-time status using WebSocket or periodic polling) and clear result visualization (confidence, highlighted frames).
- Deliverable: Frontend with robust upload and progress UX integrated with backend endpoints.

5. Model Compression & Acceleration

- Rationale: Reduce model size and inference time to enable deployment on limited-resource servers or edge devices.
- Implementation steps:
    - Apply pruning, quantization-aware training, and/or post-training quantization (TensorFlow Model Optimization Toolkit) to ResNeXt backbone.
    - Convert models to TFLite or ONNX for edge compatibility and measure accuracy drop vs gains in latency.
- Deliverable: Compressed model variants and a short report on trade-offs (size, latency, accuracy).

6. Improved Data Pipeline: Augmentation & ComImp Usage

- Rationale: Improve generalizability by using aggressive augmentation and dataset combination techniques.
- Implementation steps:
    - Add realistic augmentations: compression (variable bitrates), frame-rate changes, color jitter, occlusion, and noise.
    - Use ComImp (or a simplified vertical-merging imitation) to merge smaller internal datasets for fine-tuning if multiple labeled sets are available.

    - Implement cross-dataset evaluation and report results.
- Deliverable: Augmented training pipeline and reproducible cross-dataset evaluation scripts.

7. Explainability (XAI) — Grad-CAM for Frame-Level Insights

- Rationale: Add trust by showing which facial regions or frames contributed to a 'fake' verdict.
- Implementation steps:
    - Integrate Grad-CAM/Integrated Gradients for the CNN backbone and overlay heatmaps on key frames.
    - Expose these visualizations in the frontend as an optional "Why flagged?" view.
- Deliverable: Basic XAI visualization for selected frames in the result page.

8. Privacy & Data Handling Improvements

- Rationale: Ensure user confidence and meet basic privacy expectations.
- Implementation steps:
    - Enforce HTTPS for all communications; delete uploaded videos after analysis; keep only transient logs for debugging.
    - Optionally support client-side face-cropping before upload to reduce data transferred and stored.
- Deliverable: Privacy policy checklist implemented and configuration for secure handling.

9. CI/CD, Testing & Documentation

- Rationale: Improve reproducibility and ease future maintenance.
- Implementation steps:
    - Add unit tests for preprocessing and inference pipeline; integration tests for the /analyze_video endpoint.
    - Set up simple CI (GitHub Actions) to run tests on push and deploy a staging container.
    - Provide end-to-end README and usage docs for reproducibility.
- Deliverable: Test suite, CI pipeline, and user/developer documentation.

10. Lightweight On-Device Prototype (Stretch Goal)

- Rationale: Provide proof-of-concept that compressed models can run on a mid-range smartphone (Android) for offline inference.
- Implementation steps:
    - Convert compressed model to TFLite; build a minimal Android app to feed pre-recorded short clips and display verdicts.

    - Measure latency and accuracy on sample clips.
- Deliverable: A small demo APK (proof-of-concept) or a detailed implementation plan if full prototyping is not feasible.

References

1. MesoNet: a Compact Facial Video Forgery Detection Network https://ieeexplore.ieee.org/document/8630761
2. "Recurrent Convolutional Strategies for Face Manipulation Detection in Videos" published in CVPR Workshops, 2019 https://openaccess.thecvf.com/content_CVPRW_2019/papers/Media%20Forensics/Sabir_Recurrent_Convolutional_Strategies_for_Face_Manipulation_Detection_in_Videos_CVPRW_2019_paper.pdf
3. "Deepfake Video Detection Through Weight-Based Ensemble Learning" published in IEEE Transactions on Information Forensics and Security, 2023 https://link.springer.com/article/10.1007/s11760-025-04716-1
4. "Deepfake video detection: challenges and opportunities" Published in Springer (Artificial Intelligence Review, 2024) https://www.researchgate.net/publication/380973861_Deepfake_video_detection_challenges_and_opportunities
5. "Combining Datasets Vertically via Imputation (ComImp)" Published in IEEE IJCNN (International Joint Conference on Neural Networks), 2023 https://www.researchgate.net/publication/364530779_Combining_datasets_to_increase_the_number_of_samples_and_improve_model_fitting
6. Rössler, A., Cozzolino, D., Verdoliva, L., Riess, C., Thies, J., & Nießner, M., "FaceForensics++: Learning to Detect Forged Facial Images," in Proc. 2019 IEEE/CVF International Conference on Computer Vision (ICCV), Seoul, Korea (South), Oct. 2019, pp. 1–11. https://arxiv.org/abs/1901.08971
7. Güera, D., & Delp, E. J., "Deepfake Video Detection Using Recurrent Neural Networks," in Proc. 2018 15th IEEE International Conference on Advanced Video and Signal Based Surveillance (AVSS), Auckland, New Zealand, Nov. 2018. https://arxiv.org/abs/1805.02499
8. Afchar, D., Nozick, V., Yamagishi, J., & Echigo, T., "MesoNet: a Compact Facial Video Forgery Detection Network," in Proc. 2018 IEEE International Workshop on Information Forensics and Security (WIFS), Hong Kong, China, Dec. 2018. https://arxiv.org/abs/1809.00888
9. M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed

Systems," arXiv preprint arXiv:1603.04467, 2016.
https://arxiv.org/abs/1603.04467

10. Flask Documentation, "Flask Framework," Pallets.
https://flask.palletsprojects.com/en/stable/

11. OpenCV Team, "Open Source Computer Vision Library (OpenCV),"
https://opencv.org/

12. Harris, C.R., Millman, K.J., van der Walt, S.J. et al. "Array programming with NumPy," Nature 585, 357–362 (2020).
https://www.nature.com/articles/s41586-020-2649-2

13. Agarwal, Shruti et al. Two-Stream CNNs for Tampered Face Detection. CVPR Workshops, 2019.

14. Rössler et al., FaceForensics++ (ICCV 2019)

15. Tschiatschek et al., 2020